Homework 3: Neural Networks

Due: Monday, March 31 at 2:50 PM

- Start early and ask for help if needed! The training process for neural networks is often slow, so plan accordingly!
- You may discuss this assignment with your classmates. However, all discussion should be kept at a conceptual level, and code must not be shared. Violation of this policy is considered a violation of the course's academic integrity policy.
- Submitted code **must** be your own work. Any assignment that contains code or writing that is not your own (including from classmates, AI, or other external sources) will automatically receive a 0.
- If you use any sources other than the course notes, course readings, or official Python documentation, please cite them. A comment at the top of your code is sufficient.
- Submit your assignment on Gradescope. Please read the submissions section for instructions about what files are needed.

In this assignment, you will be working with PyTorch to build neural networks that can complete an image classification task. The goal of this assignment is to give you hands on experience building, optimizing, and comparing neural networks on a real data set.

CIFAR-10

For this homework, you will be working with the CIFAR-10 dataset, a commonly used image recognition data set. This data set contains 60000 images across 10 classes, representing common animals and items. Each image is a 32x32 color image. A list of classes and more information about the data can be found at https://www.cs.toronto.edu/~kriz/cifar.html. (Keep in mind that you only need to look at information about the CIFAR-10 data set, not the more complex CIFAR-100 data set.)

CIFAR-10 is included in the torchvision package, under torchvision.datasets.CIFAR10. Like other datasets included in PyTorch, it has already been split into training and test sets, which can be downloaded and accessed using the same approach as in lab 3. Each individual image should be represented as a 32x32x3 tensor.

Building Networks

After downloading the CIFAR-10 data set, use PyTorch data loaders to split it into appropriately sized minibatches. (I recommend size 64 as a starting point.) Then, train **three** different neural networks. All three networks must have different architectures, and each one must have at least two hidden layers. At least one network must be a feed-forward network (i.e. only fully connected layers) and at least one network must incorporate convolutional layers.

Beyond the requirements listed above, you have a large amount of flexibility on this assignment. Listed below are some elements that you may want to experiment with when creating your different architectures:

- Total number of layers
- Size of layers
- Number of fully-connected layers
- Number of convolutional layers
- Number/position of pooling layers
- Activation function
- Use of dropout

In general, anything that changes the structure of your network is considered a change to the architecture. Hyperparameters that do not change the structure of the network, such as the minibatch size or learning rate, are **not** considered changes to the architecture (although you may wish to experiment with these as well.)

At the end of each epoch of training, compute loss over the entire training set. (Be sure to do this with your model set to be in evaluation/no gradient mode.) If, at the end of an epoch, loss has increased compared to the previous epoch, halt training. You may additionally set a minimum and/or maximum number of epochs, in order to prevent overly short or long training processes. You do **not** need to create a validation set, and you should not use the test set at all during this phase.

Please note that the CIFAR-10 data set is quite large and deeper networks may take some time to train, especially if your computer does not have a GPU. While you may be able to mitigate these problems by using smaller minibatches or layer sizes, please be sure to give yourself enough time to fully train both models! I also encourage saving the model after each epoch.

Testing the Network

Once your networks have been trained, test them on the provided test set and calculate the accuracy. For each network, report the following:

1. A brief written description of the network architecture. This should include enough details that I can distinguish the difference between your networks.
2. The final accuracy of your model on the test set.
3. A graph of the training loss after each epoch of training.
4. One image from the test set that was correctly classified by your network and one that was incorrectly classified. For both images, show the image and state both the true and predicted labels. (Please be sure to give the human-readable label, e.g. “bird”, not 2.)

Finally, compare all three networks. State which network performed the best and why you think it performed the best, as well as which network performed the worst and why you think it performed the worst.

Submission and Grading

Turn in all your code in a Python file called cifar.py. This should include the code to build and train both networks. Additionally, turn in your results and analysis in a PDF file called results.pdf.

This assignment does not have any starter code and will not use an autograder. However, your submitted code does need to contain all code written to both train and test all three networks. You will be (manually) graded based on how well you use PyTorch to train and test your networks, how well you present and analyze your results, and whether you chose appropriate network architectures.

A note on network performance and grading: You will **not** be graded on the performance of your model, as long as you have trained the network properly, you are using a reasonable architecture, and you accurately report your results. This is to ensure that all students can complete this assignment regardless of what computer they are using, as it should be possible to train a small (but not trivially small) network even on an older or slower computer. However, I highly encourage you to aim for an accuracy of at least 50% for feed-forward networks and 70% for convolutional networks. (State of the art results for this data set are over 99% accuracy, and over 80% accuracy is possible even with a relatively small CNN.)